Now we write the model

Last time: inspect what the weights learned.
This time: implement the computation and train it.



Token IDs → **Embeddings** → **Attention** → **Final scores**

integers | vectors | retrieve | 36 choices

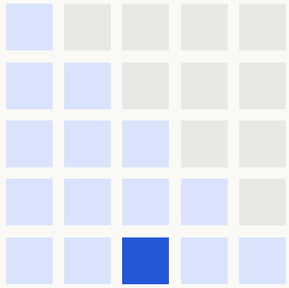
Implement. Train. Inspect what changed.

Attention on a GPU

$$A = \mathrm{softmax}(QK^T / \sqrt{d_k} + M) \qquad O = AV$$

One score is a dot product

$$s_{ij} = q_{i1}k_{j1} + q_{i2}k_{j2} + \dots$$



Many positions. Many examples.
Many dot products.

Parallel work

Which threads compute each part?

Memory

Which numbers can we reuse?

Coordination

When can the next operation read?

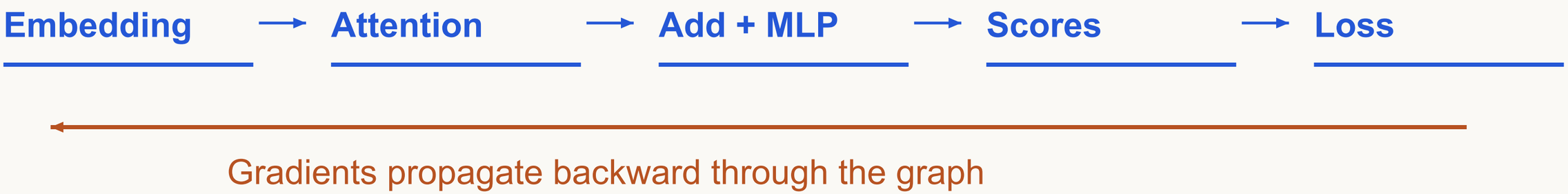
Numerics

How do masking and softmax stay correct?

Fast attention also reduces memory traffic. FlashAttention reorganizes exact attention around this cost.

Training adds a backward pass

A whole model needs a prediction, an error measure, and derivatives through the computation.



Automatic differentiation
Calculate how the loss changes with each parameter.

Optimizer
Use those gradients to update the parameters.

Tensors describe the computation

Example: 16 independent inputs, 9 positions each, 32 features per position.

OBJECT	SHAPE	MEANING
Token IDs	(16, 9)	One integer per token
Embedded inputs	(16, 9, 32)	One vector per position
Attention scores	(16, 9, 9)	Queries compare with source positions
Final-position logits	(16, 36)	One score per possible output token
Mean loss	()	One scalar objective

Shape tells you the axes. **Data type** tells you the numbers. **Device** tells you where.

Check: with 10 positions, what shape do attention scores have?

Attention in four tensor operations

The learned projections have produced q, k, and v. The causal mask is already built.

```
scores = q @ k.transpose(-1, -2)
scores = scores / math.sqrt(q.shape[-1])
weights = (scores + mask).softmax(axis=-1)
write = weights @ v
```

Compare

Scale

Mask + normalize

Retrieve

q, k, v: (B, T, 32)
scores, weights: (B, T, T)
write: (B, T, 32)

Allowed: mask = 0
Future: mask = -∞
Each attention row sums to 1.

Tensor code becomes executable work

Tensor expressions
what you write

↓

Computation graph
operations and dependencies

↓

Scheduling + lowering
divide and translate the work

↓

Executable kernels
compiled numerical programs

↓

CPU or GPU
the chosen device

tinygrad
Builds a lazy graph, schedules work, generates code, and executes it.

PyTorch
Eager operations dispatch to implementations. Optional compilation can optimize larger regions.

One Python line need not mean one kernel.

Our first lab device is CPU. The tensor mathematics stays the same.

The training loop

One update in the supplied harness, using tinygrad 0.14.0:

```
with Context(TRAINING=1):
    optimizer.zero_grad()
    logits = model(tokens)
    loss = logits.sparse_categorical_crossentropy(targets)
    loss.backward()
    optimizer.step()
```

backward() calculates gradients.
step() changes parameters.

Only the final position is graded.
Cross-entropy takes scores directly.

Predict. Measure error. Calculate gradients. Update.

Five experiments, shared components

TASK	LAYERS	HEADS / LAYER	POSITIONS	MLP
0 Hidden digit	0	-	No	No
1 Hidden digit	1	1	No	No
2 Dictionary lookup	2	1	Yes	No
3 Two-lookup sum	2	1	Yes	After layer 2
4 Two-lookup difference	2	2	Yes	After layer 2

Every run starts with fresh weights and an optimizer. Residual width: 32. Head width: 32.

Task 0 should fail. One head can sometimes solve subtraction; two heads made it more reliable across seeds.

Your local project

Python + tinygrad, on your own Windows or Mac computer.

01	**Get the repository**	Your fork and local working folder
02	**Create .venv**	An isolated Python environment
03	**Install and verify**	Pinned package, compiler, numerical check
04	**Open the first exercise**	Continue through the lab checkpoints

Today we begin. The lab continues across sessions.

github.com/rvfvn/transformer-training-lab

Sources: [13] [14] Full links on pages 14-15

Windows and Mac setup

Run these from the repository folder. Full prerequisite instructions are in README.md.

Windows PowerShell

```
py -3.12 --version
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install `
  tinygrad==0.14.0
clang --version
```

macOS Terminal

```
python3.12 --version
python3.12 -m venv .venv
.venv/bin/python -m pip install \
  tinygrad==0.14.0
clang --version
```

Use the .venv interpreter in your editor.

Run check_environment.py with that same Python.

CPU execution requires a working compiler/backend. Native Windows setup needs a separate instructor rehearsal.

Three checks before training

01 **Compute**

Sum the squares of 1, 2, and 3.

02 **Differentiate**

Check the gradient: twice each input.

03 **Update**

Use SGD to change a parameter.

Expected values

```
device: CPU

sum of squares: 14.0

gradient: [2.0, 4.0, 6.0]

parameter after update: 2.600

SETUP OK
```

An import check alone is insufficient. Use the complete diagnostic in the README.

Configuration is part of the experiment

Environment

Python version

tinygrad version

Device and compiler

Model

Layers and heads

Residual and head width

Positions and MLP

Training

Learning rate

Batch size and steps

Seeds and held-out data

Example model settings

```
"n_layers": 2,  "n_heads": 1,  "head_dim": 32,
"position_embeddings": true,  "mlp_layers": [1]
```

Layer index 1 means the second layer. Change one setting, predict, run, and record.

Save, explain, submit

Class repo → **Your fork** → **Your branch** → **Your changes**

Pull request: propose your changes back to the class repository

---

Include

Source code, saved configurations,
small results report, explanations.

Exclude

.venv, caches, credentials,
unrequested large checkpoints.

```
Base: rvfvn/transformer-training-lab / main
Head: your fork / lab/YOUR-HANDLE
```

Submit at the course checkpoint. There is no first-session completion requirement.

Sources: [17] [18] Full links on pages 14-15

Sources: concepts

Click a title or address to open the primary source.

[01] **Previous lecture: One head at a time**
https://boopdotpng.github.io/slide-decks/lec2-interp/#1

[02] **NVIDIA: Writing SIMT kernels**
https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html

[03] **Dao et al.: FlashAttention (2022)**
https://arxiv.org/abs/2205.14135

[04] **PyTorch: Tensors**
https://docs.pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html

[05] **PyTorch: Automatic differentiation**
https://docs.pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html

[06] **PyTorch: Scaled dot-product attention**
https://docs.pytorch.org/docs/2.9/generated/torch.nn.functional.scaled_dot_product_attention.html

[07] **tinygrad: Tensor operations**
https://docs.tinygrad.org/tensor/ops/

[08] **tinygrad: Movement and transpose**
https://docs.tinygrad.org/tensor/movement/

[09] **tinygrad: Scheduling, lowering, execution**
https://docs.tinygrad.org/developer/developer/

[10] **PyTorch: Introduction to torch.compile**
https://docs.pytorch.org/tutorials/intermediate/torch_compile_tutorial.html

Sources: local workflow

Click a title or address to open the primary source.

[11] **tinygrad: Quickstart and training**
https://docs.tinygrad.org/quickstart/

[12] **tinygrad v0.14.0: Optimizer implementation**
https://github.com/tinygrad/tinygrad/blob/v0.14.0/tinygrad/nn/optim.py

[13] **Python 3.12: Virtual environments**
https://docs.python.org/3.12/library/venv.html

[14] **PyPI: tinygrad 0.14.0**
https://pypi.org/project/tinygrad/0.14.0/

[15] **tinygrad: Runtime requirements**
https://docs.tinygrad.org/runtime/

[16] **tinygrad: Device and environment settings**
https://docs.tinygrad.org/env_vars/

[17] **GitHub: Contributing to a project**
https://docs.github.com/en/get-started/exploring-projects-on-github/contributing-to-a-project

[18] **GitHub: Create a pull request from a fork**
https://docs.github.com/en/pull-requests/how-tos/create-pull-requests/creating-a-pull-request-from-a-fork

[19] **Apple: Installing the command-line tools**
https://developer.apple.com/documentation/xcode/installing-the-command-line-tools/

[20] **LLVM: Official releases**
https://github.com/llvm/llvm-project/releases